kimi-k2-thinking-hf-model-card

---

license: other license_name: modified-mit library_name: transformers



Chat Kimi K2 Homepage Moonshot AI
Hugging Face Moonshot AI Twitter Kimi.ai Discord Kimi.ai
License Modified MIT

Tech Blog

1. Model Introduction

Kimi K2 Thinking is the latest, most capable version of open-source thinking model. Starting with Kimi K2, we built it as a thinking agent that reasons step-by-step while dynamically invoking tools. It sets a new state-of-the-art on Humanity's Last Exam (HLE), BrowseComp, and other benchmarks by dramatically scaling multi-step reasoning depth and maintaining stable tool-use across 200–300 sequential calls. At the same time, K2 Thinking is a native INT4 quantization model with 256k context window, achieving lossless reductions in inference latency and GPU memory usage.

Key Features

- **Deep Thinking & Tool Orchestration**: End-to-end trained to interleave chain-of-thought reasoning with function calls, enabling autonomous research, coding, and writing workflows that last hundreds of steps without drift.
- **Native INT4 Quantization**: Quantization-Aware Training (QAT) is employed in post-training stage to achieve lossless 2x speed-up in low-latency mode.
- **Stable Long-Horizon Agency**: Maintains coherent goal-directed behavior across up to 200–300 consecutive tool invocations, surpassing prior models that degrade after 30–50 steps.

2. Model Summary

Architecture	Mixture-of-Experts (MoE)
Total Parameters	1T
Activated Parameters	32B
Number of Layers (Dense layer included)	61
Number of Dense Layers	1
Attention Hidden Dimension	7168
MoE Hidden Dimension (per Expert)	2048
Number of Attention Heads	64
Number of Experts	384
Selected Experts per Token	8
Number of Shared Experts	1
Vocabulary Size	160K

Context Length	256K
Attention Mechanism	MLA
Activation Function	SwiGLU

3. Evaluation Results

Reasoning Tasks

Benchmark	Setting	K2 Thinking	GPT-5 (High)	Claude Sonnet 4.5 (Thinking)	K2 0905	DeepSeek-V3.2	Grok-4
HLE (Text-only)	no tools	23.9	26.3	19.8*	7.9	19.8	25.4
	w/ tools	44.9	41.7*	32.0*	21.7	20.3*	41.0
	heavy	51.0	42.0	-	-	-	50.7
AIME25	no tools	94.5	94.6	87.0	51.0	89.3	91.7
	w/ python	99.1	99.6	100.0	75.2	58.1*	98.8
	heavy	100.0	100.0	-	-	-	100.0
HMMT25	no tools	89.4	93.3	74.6*	38.8	83.6	90.0
	w/ python	95.1	96.7	88.8*	70.4	49.5*	93.9
	heavy	97.5	100.0	-	-	-	96.7
IMO-AnswerBench	no tools	78.6	76.0*	65.9*	45.8	76.0*	73.1
GPQA	no tools	84.5	85.7	83.4	74.2	79.9	87.5

General Tasks

Benchmark	Setting	K2 Thinking	GPT-5 (High)	Claude Sonnet 4.5 (Thinking)	K2 0905	DeepSeek-V3.2
MMLU-Pro	no tools	84.6	87.1	87.5	81.9	85.0
MMLU-Redux	no tools	94.4	95.3	95.6	92.7	93.7
Longform Writing	no tools	73.8	71.4	79.8	62.8	72.5
HealthBench	no tools	58.0	67.2	44.2	43.8	46.9

Agentic Search Tasks

Benchmark	Setting	K2 Thinking	GPT-5 (High)	Claude Sonnet 4.5 (Thinking)	K2 0905	DeepSeek-V3.2
BrowseComp	w/ tools	60.2	54.9	24.1	7.4	40.1
BrowseComp-ZH	w/ tools	62.3	63.0*	42.4*	22.2	47.9
Seal-0	w/ tools	56.3	51.4*	53.4*	25.2	38.5*
FinSearchComp-T3	w/ tools	47.4	48.5*	44.0*	10.4	27.0*
Frames	w/ tools	87.0	86.0*	85.0*	58.1	80.2*

Coding Tasks

Benchmark	Setting	K2 Thinking	GPT-5 (High)	Claude Sonnet 4.5 (Thinking)	K2 0905	DeepSeek-V3.2
SWE-bench Verified	w/ tools	71.3	74.9	77.2	69.2	67.8

Benchmark	Setting	K2 Thinking	GPT-5 (High)	Claude Sonnet 4.5 (Thinking)	K2 0905	DeepSeek-V3.2
SWE-bench Multilingual	w/ tools	61.1	55.3*	68.0	55.9	57.9
Multi-SWE-bench	w/ tools	41.9	39.3*	44.3	33.5	30.6
SciCode	no tools	44.8	42.9	44.7	30.7	37.7
LiveCodeBenchV6	no tools	83.1	87.0*	64.0*	56.1*	74.1
OJ-Bench (cpp)	no tools	48.7	56.2*	30.4*	25.5*	38.2*
Terminal-Bench	w/ simulated tools (JSON)	47.1	43.8	51.0	44.5	37.7

▶ **Footnotes**

4. Native INT4 Quantization

Low-bit quantization is an effective way to reduce inference latency and GPU memory usage on large-scale inference servers. However, thinking models use excessive decoding lengths, and thus quantization often results in substantial performance drops.

To overcome this challenge, we adopt Quantization-Aware Training (QAT) during the post-training phase, applying INT4 weight-only quantization to the MoE components. It allows K2 Thinking to support native INT4 inference with a roughly 2x generation speed improvement while achieving state-of-the-art performance. All benchmark results are reported under INT4 precision.

The checkpoints are saved in compressed-tensors format, supported by most of mainstream inference engine. If you need the checkpoints in higher precision such as FP8 or BF16, you can refer to official repo of compressed-tensors to unpack the int4 weights and convert to any higher precision.

5. Deployment

> [!Note] You can access K2 Thinking's API on https://platform.moonshot.ai , we provide OpenAI/Anthropic-compatible API for you.

Currently, Kimi-K2-Thinking is recommended to run on the following inference engines:

- vLLM
- SGLang
- KTransformers

Deployment examples can be found in the Model Deployment Guide.

---

6. Model Usage

Chat Completion

Once the local inference service is up, you can interact with it through the chat endpoint:

```
def simple_chat(client: openai.OpenAI, model_name: str):
    messages = [
```

```
        {"role": "system", "content": "You are Kimi, an AI assistant created by Moonshot
        AI."},
        {"role": "user", "content": [{"type": "text", "text": "which one is bigger, 9.11 or
        9.9? think carefully."}]},
    ]
    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        temperature=1.0,
        max_tokens=4096
    )
    print(f"k2 answer: {response.choices[0].message.content}")
    print("=====below is reasoning content======")
    print(f"reasoning content: {response.choices[0].message.reasoning_content}")
```

[!NOTE] The recommended temperature for Kimi-K2-Thinking is `temperature = 1.0`. If no special instructions are required, the system prompt above is a good default.

Tool Calling

Kimi-K2-Thinking has the same tool calling settings as Kimi-K2-Instruct.

To enable them, you need to pass the list of available tools in each request, then the model will autonomously decide when and how to invoke them.

The following example demonstrates calling a weather tool end-to-end:

```python
# Your tool implementation
def get_weather(city: str) -> dict:
    return {"weather": "Sunny"}
# Tool schema definition
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Retrieve current weather information. Call this when the user asks
        about the weather.",
        "parameters": {
            "type": "object",
            "required": ["city"],
            "properties": {
                "city": {
                    "type": "string",
                    "description": "Name of the city"
                }
            }
        }
    }
}]
# Map tool names to their implementations
tool_map = {
    "get_weather": get_weather
}
def tool_call_with_client(client: OpenAI, model_name: str):
    messages = [
        {"role": "system", "content": "You are Kimi, an AI assistant created by Moonshot
        AI."},
```

```
        {"role": "user", "content": "What's the weather like in Beijing today? Use the tool
        to check."}
    ]
    finish_reason = None
    while finish_reason is None or finish_reason == "tool_calls":
        completion = client.chat.completions.create(
            model=model_name,
            messages=messages,
            temperature=1.0,
            tools=tools,          # tool list defined above
            tool_choice="auto"
        )
        choice = completion.choices[0]
        finish_reason = choice.finish_reason
        if finish_reason == "tool_calls":
            messages.append(choice.message)
            for tool_call in choice.message.tool_calls:
                tool_call_name = tool_call.function.name
                tool_call_arguments = json.loads(tool_call.function.arguments)
                tool_function = tool_map[tool_call_name]
                tool_result = tool_function(**tool_call_arguments)
                print("tool_result:", tool_result)
                messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "name": tool_call_name,
                    "content": json.dumps(tool_result)
                })
    print("-" * 100)
    print(choice.message.content)
```

The `tool_call_with_client` function implements the pipeline from user query to tool execution. This pipeline requires the inference engine to support Kimi-K2's native tool-parsing logic. For more information, see the Tool Calling Guide.

---

7. License

Both the code repository and the model weights are released under the Modified MIT License.

---

8. Third Party Notices

See THIRD PARTY NOTICES

---

9. Contact Us

If you have any questions, please reach out at support@moonshot.cn.